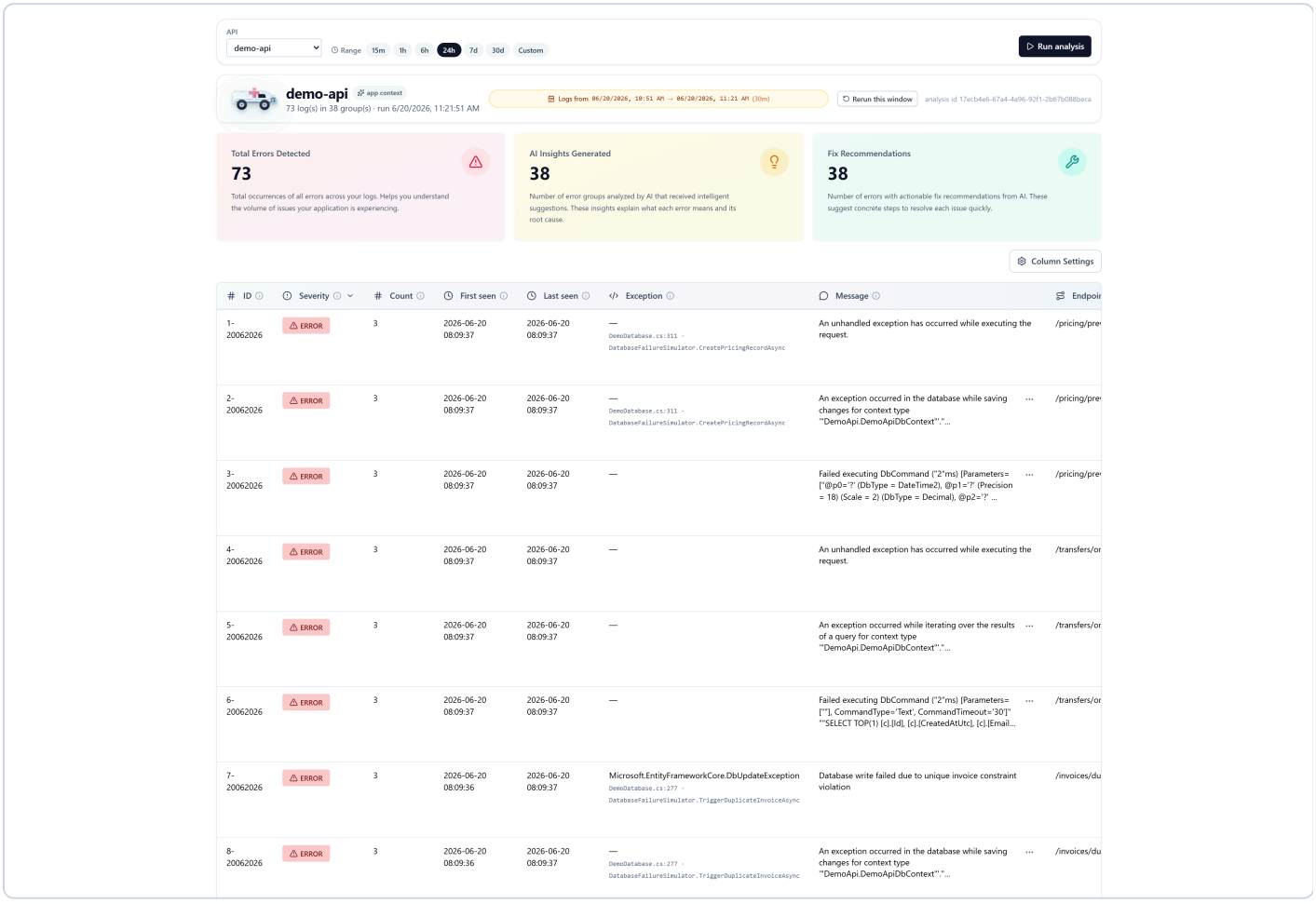
HotFixAmbulance — Working System Evidence

Production-error triage for .NET Web APIs, with AI analysis by a locally-hosted **Qwen** model. Captured 2026-06-20.

What it does & how it works. HotFixAmbulance pulls the last N hours of error logs from **Elasticsearch**, groups them by exception fingerprint, and for every group asks a locally-hosted **Qwen LLM (** `qwen2.5:3b` **)** — served in **Docker** through an Ollama-compatible runtime — to write a plain-English *"Suggestion for Error"* (what the error means) and a concrete *"How to fix"* (the remediation), grounded in the service's Git history. Each run is persisted to **SQLite** and rendered in a **React** UI. Every group the model analyses is tagged `analyzedBy = "Llm"`, and the UI renders a violet Qwen badge on its two AI columns — so the use of Qwen is visible and verifiable, per group.

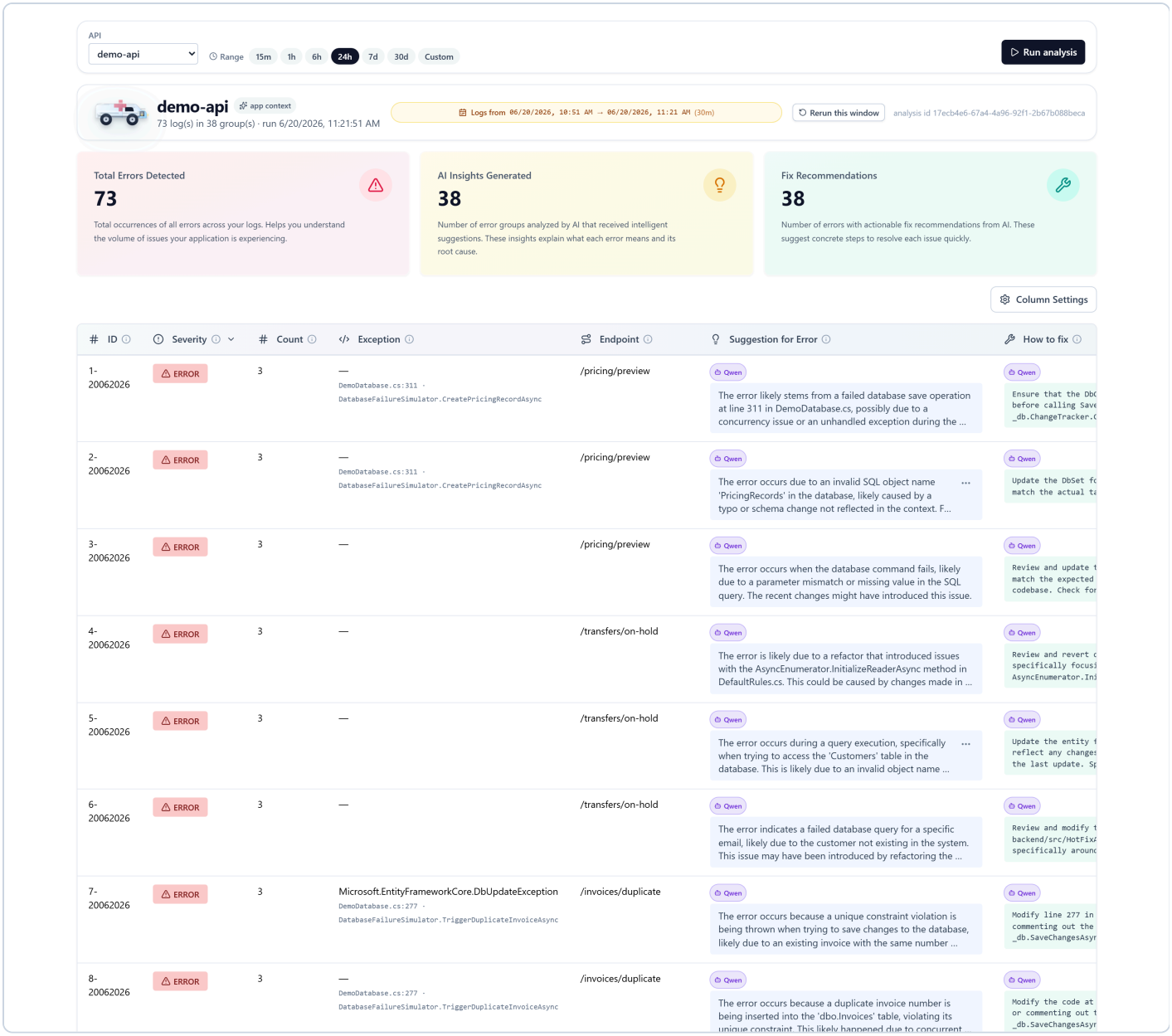
Pipeline: Elasticsearch → group by fingerprint → per-group Qwen enrichment (with Git-history grounding) → SQLite → REST API (`:5283`) → React UI (`:5173`).
Runtime: Qwen in Docker (CPU-only, `:11434`).

Figure 1 — System overview (ingestion, grouping, AI layer ran)



Triage run for the **demo-api** service: **73 error logs** collapsed into **38 distinct groups**. The metrics panel confirms the AI layer ran end-to-end — **38 "AI Insights Generated"** and **38 "Fix Recommendations"**, i.e. *every* group was enriched. The table shows each group's severity, occurrence count, first/last-seen, exception type with stack frame, message and endpoint — the raw evidence read from Elasticsearch. **Proves:** the ingestion → grouping → analysis pipeline works end-to-end.

Figure 2 — Proof the analysis is produced by Qwen



The two right-most columns are the AI columns. Every cell carries a violet Qwen badge, which the UI renders *only* when a group's `analyzedBy === 'llm'` — a value the backend sets *exclusively* after a successful Qwen completion (any model failure silently falls back to the heuristic and shows no badge). The text is genuinely model-generated and context-specific — e.g. *"The error likely stems from a failed database save operation at line 311 in DemoDatabase.cs…"* and *"The error occurs due to an invalid SQL object name 'Pricing.Records'…"* — not a static template. **Proves:** the system demonstrably uses Qwen to analyse each error and propose a fix.

Figure 3 — The complete result set (all 38 groups)

API

demo-api | Range | 15m | 1h | 6h | 24h | 7d | 30d | Custom | Run analysis

demo-api app context

73 log(s) in 38 group(s) · run 6/20/2026, 11:21:51 AM

Logs from 06/20/2026, 10:51 AM → 06/20/2026, 11:21 AM (30m) | Rerun this window | analysis id 17ecb4e6-67a4-4a96-92f1-2b67b088beca

Total Errors Detected

73

Total occurrences of all errors across your logs. Helps you understand the volume of issues your application is experiencing.

AI Insights Generated

38

Number of error groups analyzed by AI that received intelligent suggestions. These insights explain what each error means and its root cause.

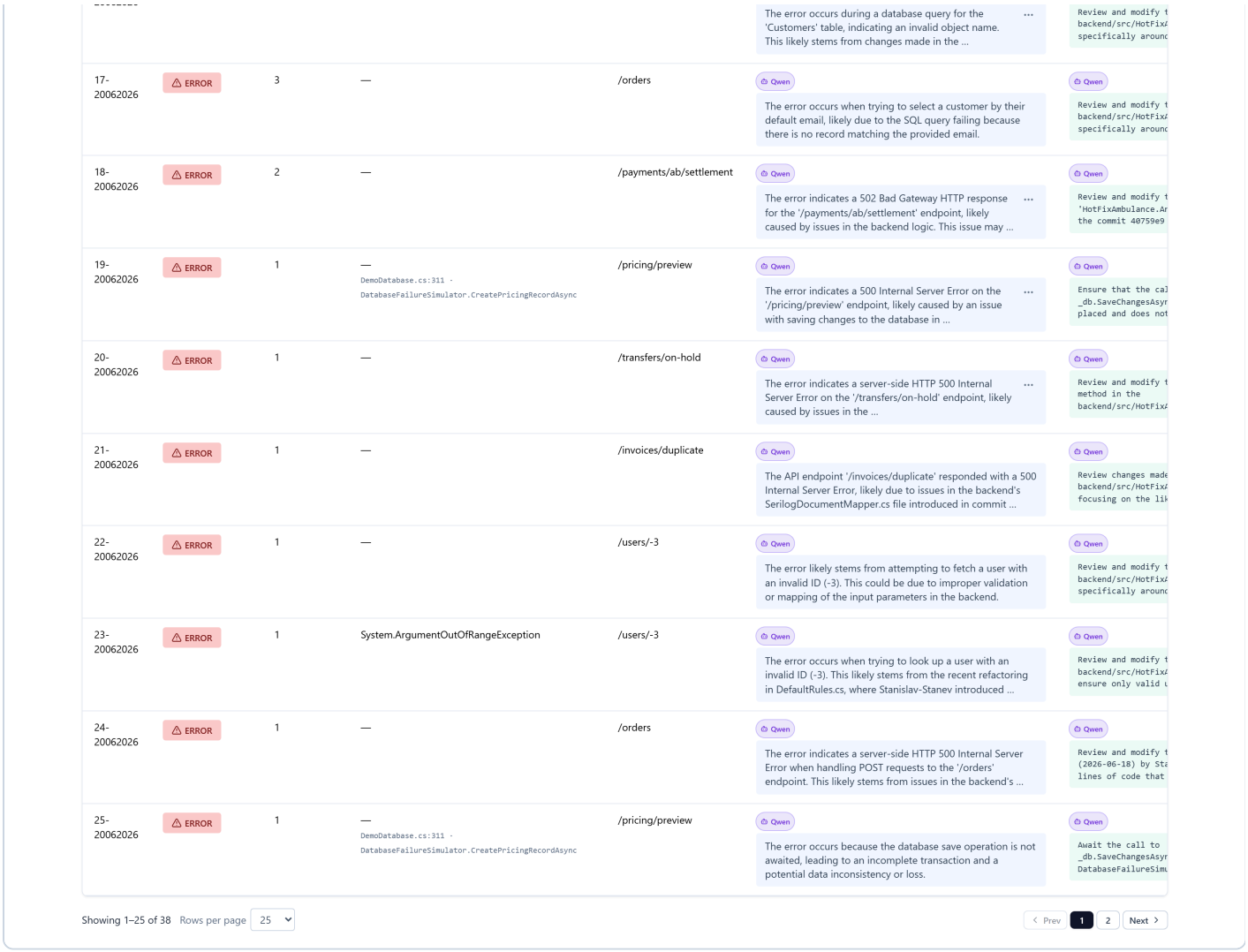
Fix Recommendations

38

Number of errors with actionable fix recommendations from AI. These suggest concrete steps to resolve each issue quickly.

Column Settings

ID	Severity	Count	Exception	Endpoint	Suggestion for Error	How to fix
1-20062026	ERROR	3	— DemoDatabase.cs:311 · DatabaseFailureSimulator.CreatePricingRecordAsync	/pricing/preview	Qwen The error likely stems from a failed database save operation at line 311 in DemoDatabase.cs, possibly due to a concurrency issue or an unhandled exception during the ...	Qwen Ensure that the DbC before calling Save _db.ChangeTracker.C
2-20062026	ERROR	3	— DemoDatabase.cs:311 · DatabaseFailureSimulator.CreatePricingRecordAsync	/pricing/preview	Qwen The error occurs due to an invalid SQL object name 'PricingRecords' in the database, likely caused by a typo or schema change not reflected in the context. F...	Qwen Update the DbSet fo match the actual ta
3-20062026	ERROR	3	—	/pricing/preview	Qwen The error occurs when the database command fails, likely due to a parameter mismatch or missing value in the SQL query. The recent changes might have introduced this issue.	Qwen Review and update t match the expected codebase. Check for
4-20062026	ERROR	3	—	/transfers/on-hold	Qwen The error is likely due to a refactor that introduced issues with the AsyncEnumerator.InitializeReaderAsync method in DefaultRules.cs. This could be caused by changes made in ...	Qwen Review and revert c specifically focusi AsyncEnumerator.Ini
5-20062026	ERROR	3	—	/transfers/on-hold	Qwen The error occurs during a query execution, specifically when trying to access the 'Customers' table in the database. This is likely due to an invalid object name ...	Qwen Update the entity f reflect any changes the last update. Sp
6-20062026	ERROR	3	—	/transfers/on-hold	Qwen The error indicates a failed database query for a specific email, likely due to the customer not existing in the system. This issue may have been introduced by refactoring the ...	Qwen Review and modify t backend/src/HotFixA specifically around
7-20062026	ERROR	3	Microsoft.EntityFrameworkCore.DbUpdateException DemoDatabase.cs:277 · DatabaseFailureSimulator.TriggerDuplicateInvoiceAsync	/invoices/duplicate	Qwen The error occurs because a unique constraint violation is being thrown when trying to save changes to the database, likely due to an existing invoice with the same number ...	Qwen Modify line 277 in commenting out the _db.SaveChangesAsyn
8-20062026	ERROR	3	— DemoDatabase.cs:277 · DatabaseFailureSimulator.TriggerDuplicateInvoiceAsync	/invoices/duplicate	Qwen The error occurs because a duplicate invoice number is being inserted into the 'dbo.Invoices' table, violating its unique constraint. This likely happened due to concurrent ...	Qwen Modify the code at or commenting out t _db.SaveChangesAsyn
9-20062026	ERROR	3	Microsoft.Data.SqlClient.SqlException	/payments/ab/settlement	Qwen The error occurs when the system fails to find a payment record by ID 'ab'. This likely stems from changes in the backend code, specifically around Entity Framework ...	Qwen Review and update t backend/src/HotFixA focusing on the Asy
10-20062026	ERROR	3	—	/payments/ab/settlement	Qwen The error occurs during a query execution, specifically when trying to read from the 'Payments' table. This likely stems from changes made in the ...	Qwen Review and update t HotFixAmbulance.Ana correctly reference
11-20062026	ERROR	3	—	/payments/ab/settlement	Qwen The error indicates a failed database query, likely due to the PaymentId parameter not matching any records in the Payments table. This issue may have been introduced by ...	Qwen Review and modify t 40759e9 (2026-06-18 parameter is correc
12-20062026	ERROR	3	Microsoft.Data.SqlClient.SqlException	/payments/ab	Qwen The error occurs when trying to fetch a payment by ID 'ab', likely due to the recent refactoring in Phase 10.1, which may have introduced issues.	Qwen Review and update t backend/src/HotFixA AsyncEnumerator.Ini
13-20062026	ERROR	3	—	/payments/ab	Qwen The error indicates an invalid object name 'Payments' in the database query for context type 'DemoApi.DemoApiDbContext'. This likely stems fro...	Qwen Update the entity c correct table name one. Review and upd
14-20062026	ERROR	3	—	/payments/ab	Qwen The error occurs when the API fails to execute a SQL query for retrieving a payment by its ID. This likely stems from changes made in the refactoring of the demo-api, possibly...	Qwen Review and update t backend/src/HotFixA specifically around
15-20062026	ERROR	3	Microsoft.Data.SqlClient.SqlException	/orders	Qwen The error indicates an invalid object name 'Customers' in the SQL query, likely caused by a change in database schema or a typo. The git history points to changes relate...	Qwen Review and update t backend/src/HotFixA the current databas
16-20062026	ERROR	3	—	/orders	Qwen	Qwen



The full run. **All 38 groups** carry the Qwen badge on both AI columns, confirming the entire run was analysed by Qwen (run-level `analyzedBy = "Llm"`), not the heuristic fallback. **Proves:** the whole demo flow runs on Qwen.

Figure 4 — Backend / runtime evidence (terminal)

```
# The Qwen model is served from Docker (CPU-only) on :11434
$ docker ps --filter name=hfa-qwen
hfa-qwen   ollama/ollama:latest   Up 59 minutes (healthy)   0.0.0.0:11434->11434/tcp

# The qwen2.5:3b model is pulled into the persistent volume
$ docker exec hfa-qwen ollama list
NAME          ID              SIZE     MODIFIED
qwen2.5:3b    357c53fb659c    1.9 GB   37 minutes ago

# A triage run produced via the API — analyzed by the LLM, every group enriched
$ curl -s http://localhost:5283/api/triage/runs/17ecb4e6-.../
apiName     = demo-api
analyzedBy  = Llm
totalLogs   = 73
totalGroups = 38
summary.withSuggestions = 38      # all 38 groups got an AI suggestion
summary.withFixes       = 38      # all 38 groups got an AI fix

# During inference the Qwen container saturates the CPU (one call per group)
$ docker stats hfa-qwen --no-stream
hfa-qwen   CPU = 1177%   MEM = 4.4 GiB / 15.4 GiB
```

Proves: Qwen runs locally in Docker, the model is loaded, and a real triage run returns `analyzedBy = Llm` with an AI suggestion + fix for every one of the 38 groups.

How the project is built (modules)

- **Ingestion** — `HotFixAmbulance.Elastic`: queries Elasticsearch for Fatal/Error/Warning logs in the window.
- **Analysis / grouping** — `HotFixAmbulance.Analysis`: groups logs by (exception, normalized message, endpoint) and ranks them.
- **AI enrichment** — `LlmGroupEnricher` + `OllamaLlmClient`: calls the Qwen runtime per group; on any failure falls back to a deterministic Git-history heuristic.
- **Git insights** — `HotFixAmbulance.GitInsights`: mines `origin/main` for blame + related commits to ground the model.
- **Persistence** — `HotFixAmbulance.Persistence`: stores each run (and per-group `AnalyzedBy`) in SQLite.
- **API + UI** — ASP.NET Core REST API and a React/TanStack table that renders the 🤖 Qwen badge.
- **Infrastructure** — `infra/qwen`: Docker Compose Qwen runtime + bootstrap that pre-pulls the model (with corporate-CA injection for TLS-intercepting proxies).

Repository: https://github.com/myPOStech/mps-banking-hot-fix-ambulance · **AI tool used:** Claude Code (Opus/Sonnet) for design, implementation, TDD and orchestration.